Requisitos del sistema

1.1. Requisitos Funcionales

Código	RF-01
Título	Generación del Árbol de Sintaxis Abstracta (AST)
Descripción	El sistema debe procesar el programa ingresado por el usuario utilizando la gramática previamente definida para generar su correspondiente Árbol de Sintaxis Abstracta (AST).
Prioridad	Alta
Criterios de aceptación	▪ El sistema invoca el intérprete Racket con los archivos del editor y la expresión de entrada; el AST resultante es serializado en formato JSON y transmitido al cliente. ▪ El AST generado es renderizado en el panel ASTViewer reflejando la estructura jerárquica del programa evaluado. ▪ Cuando el programa contiene errores sintácticos o de evaluación, el sistema muestra un mensaje de error en la consola con el prefijo × y nivel visual diferenciado en rojo; el panel ASTViewer conserva el último AST válido. ▪ El AST corresponde fielmente a la última expresión evaluada exitosamente.

Cuadro 1.1: RF-01: Generación del AST

Código	RF-02
Título	Visualización gráfica e interactiva del AST
Descripción	El sistema debe mostrar el árbol de sintaxis abstracta generado de forma gráfica e interactiva, permitiendo la exploración visual de su estructura.
Prioridad	Alta

Criterios de aceptación	▪ El panel ASTViewer renderiza la representación jerárquica del AST con nodos que indican el tipo de dato retornado por el intérprete. ▪ La visualización se actualiza automáticamente tras cada ejecución exitosa, sin necesidad de acción adicional por parte del usuario. ▪ El panel es responsivo y se adapta al tamaño del contenedor asignado dentro del layout redimensionable. ▪ En modo paso a paso, el panel refleja el AST correspondiente al paso de evaluación actualmente visualizado. ▪ No se presentan errores de renderizado ni contenido huérfano durante el ciclo de vida de la sesión.

Cuadro 1.2: RF-02: Visualización gráfica e interactiva del AST

Código	RF-03
Título	Ejecución y evaluación del programa
Descripción	El sistema debe permitir ejecutar el programa del usuario e interpretar su resultado, siguiendo el proceso de evaluación definido por la gramática y el intérprete generado.
Prioridad	Alta

Criterios de aceptación	▪ El sistema transmite los archivos del editor y la expresión de entrada a la ruta `POST /api/run` y recibe el arreglo de pasos de evaluación. ▪ El resultado de cada paso es presentado en la consola con nivel de salida correspondiente: verde (`output`) para valores, rojo (`error`) para excepciones capturadas, gris (`info`) para resultados `void`. ▪ Los errores producidos por `eopl:error` son capturados por el runtime, limpiados de trazas internas de Racket, y presentados con el prefijo `×` en color rojo. ▪ La ejecución puede ser cancelada por el usuario durante el proceso; el servidor aborta el subproceso Racket correspondiente. ▪ El tiempo máximo de ejecución del subproceso Racket es de 15 segundos; transcurrido ese tiempo, el sistema reporta el error de tiempo límite en la consola. ▪ El sistema soporta un modo de evaluación paso a paso que permite avanzar la ejecución de forma controlada mostrando el estado del AST y del ambiente en cada paso.

Cuadro 1.3: RF-03: Ejecución y evaluación del programa

Código	RF-04
Título	Representación del ambiente de ejecución
Descripción	El sistema debe representar el ambiente de ejecución como una estructura jerárquica de marcos que se actualiza en cada ejecución, permitiendo al estudiante observar la evolución del estado del programa.
Prioridad	Alta

Criterios de aceptación	▪ El panel EnvironmentPanel muestra el ambiente como una pila de marcos jerárquicos, donde cada marco lista las ligaduras activas con su nombre, valor serializado y tipo de dato inferido. ▪ El ambiente mostrado corresponde al último paso de evaluación completado; en modo paso a paso, se actualiza con cada avance del usuario. ▪ La estructura de marcos refleja fielmente la cadena de ambientes construida durante la evaluación, permitiendo al estudiante identificar el alcance léxico de cada variable. ▪ El panel permanece visible y no colapsa ante la ausencia de ligaduras en el ambiente actual.

Cuadro 1.4: RF-04: Representación del ambiente de ejecución

Código	RF-05
Título	Biblioteca de ejemplos
Descripción	El sistema debe ofrecer una biblioteca de ejemplos predefinidos orientados a los indicadores de logro identificados como base conceptual del prototipo, con el propósito de facilitar el aprendizaje mediante casos de uso concretos.
Prioridad	Media
Criterios de aceptación	▪ Los ejemplos son cargados desde el sistema de archivos del servidor durante el renderizado inicial y transmitidos al cliente como propiedades del componente orquestador. ▪ El sistema presenta la lista de ejemplos disponibles en el modal de bienvenida con nombre y descripción breve de cada uno. ▪ Al seleccionar un ejemplo, los archivos del editor son reemplazados por los archivos correspondientes sin pérdida de estado de la interfaz. ▪ Los ejemplos cubren los cinco indicadores de logro seleccionados en el diagnóstico: IL 1.1.6, IL 1.2.3, IL 1.2.6, IL 1.2.7 e IL 1.3.1. ▪ Cada ejemplo es compatible con el pipeline de gramática y puede ser ejecutado sin modificaciones adicionales.

Cuadro 1.5: RF-05: Biblioteca de ejemplos

Código	RF-06
Título	Soporte y validación de sintaxis en Racket
Descripción	El sistema debe ser capaz de reconocer la sintaxis del lenguaje Racket en el editor y validar la gramática BNF ingresada por el usuario, reportando errores de forma clara y oportuna.
Prioridad	Alta
Criterios de aceptación	▪ El editor Monaco provee resaltado de sintaxis para archivos con extensión `.rkt`. ▪ El pipeline de gramática detecta y reporta errores léxicos en la especificación BNF durante el procesamiento; los mensajes son presentados en el modal de gramática antes de generar los archivos. ▪ El sistema no genera los archivos del intérprete ni habilita la ejecución cuando la gramática BNF contiene errores léxicos o sintácticos. ▪ Los errores de evaluación producidos por el intérprete Racket son presentados en la consola sin exponer trazas internas del runtime al usuario.

Cuadro 1.6: RF-06: Soporte y validación de sintaxis en Racket

Código	RF-07
Título	Visualización de los componentes internos del intérprete
Descripción	El sistema debe permitir al estudiante visualizar los archivos que componen el intérprete generado, mostrando su estructura con distinción entre las secciones técnicas y las secciones destinadas a la implementación del estudiante.
Prioridad	Media

Criterios de aceptación	▪ El panel de edición organiza los archivos del intérprete en pestañas independientes, accesibles mediante navegación por clic. ▪ Las líneas generadas automáticamente por el pipeline son marcadas como de solo lectura y presentadas con distinción visual respecto a las líneas editables. ▪ Las líneas destinadas a la implementación del estudiante (stubs de `eval-program`, `eval-expression`, `apply-primitive`) permanecen activas y editables. ▪ No se permite la modificación de las líneas protegidas durante la sesión de trabajo.

Cuadro 1.7: RF-07: Visualización de los componentes internos del intérprete

Código	RF-08
Título	Plantilla base editable y exportación del entorno
Descripción	El sistema debe proporcionar una plantilla base del intérprete donde el estudiante pueda completar la implementación, y permitir la descarga del proyecto completo en formato `.zip` compatible con la biblioteca EOPL.
Prioridad	Alta

Criterios de aceptación	▪ Al confirmar la gramática, el pipeline genera automáticamente los tres archivos del intérprete derivados de la especificación (`grammar.rkt`, `environment.rkt` y `main.rkt`) con los stubs de las funciones a implementar y los carga en el editor, junto con el archivo de utilidades `utils.rkt` de la plantilla base, sin intervención adicional del usuario. ▪ Las secciones de infraestructura técnica del intérprete están protegidas y no son modificables por el usuario. ▪ Las secciones de implementación están identificadas con comentarios `;; TODO` que orientan al estudiante sobre qué debe completar. ▪ El sistema genera un archivo `.zip` con todos los archivos del editor en su estado actual, incluyendo las modificaciones realizadas por el usuario. ▪ El archivo descargado incluye una estructura de directorios compatible con la biblioteca EOPL y puede ser abierto directamente en DrRacket.

Cuadro 1.8: RF-08: Plantilla base editable y exportación del entorno

1.2. Requisitos No Funcionales

Código	RNF-01
Título	Usabilidad
Descripción	El sistema debe ser intuitivo y operable por estudiantes sin experiencia avanzada en compiladores, minimizando la carga cognitiva extrínseca impuesta por la interfaz.
Prioridad	Alta

Criterios de aceptación	▪ Los usuarios pueden iniciar una sesión de trabajo, definir una gramática y ejecutar un programa sin consultar documentación externa. ▪ Los mensajes de error presentados en la consola son comprensibles sin conocimiento del funcionamiento interno de Racket. ▪ La distribución de paneles sigue un orden visual coherente con el flujo de trabajo del estudiante: definición de gramática, edición del intérprete, ejecución y observación de resultados. ▪ Las secciones del editor que no deben ser modificadas están diferenciadas visualmente de forma que el estudiante pueda identificarlas sin instrucción explícita.

Cuadro 1.9: RNF-01: Usabilidad

Código	RNF-02
Título	Rendimiento
Descripción	El sistema debe completar la generación del AST y la evaluación del programa en un tiempo que no interrumpa el flujo cognitivo del estudiante durante la exploración.
Prioridad	Alta
Criterios de aceptación	▪ La transformación BNF a archivos Racket se completa en el cliente en menos de 1 segundo para gramáticas de hasta 20 producciones. ▪ La respuesta de la ruta `/api/run` para programas de complejidad habitual en la asignatura FLP se recibe en menos de 5 segundos bajo condiciones normales de red. ▪ El tiempo máximo de ejecución del subproceso Racket está limitado a 15 segundos; transcurrido ese tiempo, el sistema notifica al usuario y libera los recursos asociados. ▪ No se producen bloqueos del hilo principal del navegador durante la ejecución de la petición al servidor.

Cuadro 1.10: RNF-02: Rendimiento

Código	RNF-03

Título	Accesibilidad web
Descripción	El sistema debe ejecutarse en navegadores web modernos sin requerir instalación de software adicional, plugins ni autenticación por parte del usuario.
Prioridad	Alta
Criterios de aceptación	▪ La aplicación carga y opera correctamente en las versiones actuales de Google Chrome, Mozilla Firefox y Microsoft Edge. ▪ No se requiere instalación de extensiones, plugins ni entornos de ejecución adicionales en el equipo del usuario. ▪ El acceso al sistema no requiere registro ni autenticación de ningún tipo. ▪ La interfaz es funcional en resoluciones de pantalla iguales o superiores a 1280 × 720 píxeles.

Cuadro 1.11: RNF-03: Accesibilidad web

Código	RNF-04
Título	Mantenibilidad
Descripción	El código fuente debe estar organizado de forma modular, con separación explícita de responsabilidades, para facilitar la incorporación de mejoras futuras.
Prioridad	Media
Criterios de aceptación	▪ Las definiciones de tipos e interfaces TypeScript residen exclusivamente en el directorio `app/types/`, con importaciones explícitas desde los módulos que las consumen. ▪ Las funciones puras y utilitarios que no dependen del estado React residen en `app/lib/` y no en los archivos de componentes. ▪ El pipeline de gramática está segmentado en módulos independientes (lexer, parser, generadores) con responsabilidades no solapadas. ▪ Es posible añadir un nuevo generador de archivo Racket incorporando un módulo en `app/lib/` sin modificar los componentes de interfaz.

Cuadro 1.12: RNF-04: Mantenibilidad

Código	RNF-05
Título	Escalabilidad
Descripción	La arquitectura del sistema debe permitir incorporar nuevos ejemplos, gramáticas de mayor complejidad o características adicionales sin afectar la estabilidad de las funcionalidades existentes.
Prioridad	Media
Criterios de aceptación	▪ La incorporación de nuevos ejemplos se realiza añadiendo archivos al directorio de ejemplos del servidor, sin modificar código de componentes ni de la API. ▪ El pipeline de gramática procesa gramáticas con hasta 20 producciones sin degradación observable del rendimiento en el cliente. ▪ La adición de un nuevo panel de visualización puede realizarse como un componente React independiente integrable en el layout sin alterar el comportamiento de los paneles existentes. ▪ El sistema de contenedores Docker permite escalar el servicio de ejecución de forma independiente al servidor de la interfaz.

Cuadro 1.13: RNF-05: Escalabilidad

Bibliografía